# The Nature of Problems and Classic Puzzles

# Slide Puzzle

https://slidingtiles.com/en/puzzle/play/other/25367-3x3-puzzle#3x3

# Solving Simple Sudoku

a demonstration

https://sudoku.com

# What is a problem

Assume that you have a car

- it has blue smoke coming out of the tail pipe
- it shakes when it idles
- it has lower feul efficiency

This is a *problem* that can be **solved** with automotive knowledge

If you tell your freind, they might say

> Get a new car, problem solved

But would you say that's a *solution* to the problem

# Constraints

Problems include *constraints*, unbreakable *rules* about the problem or the way it should be solved

With the car example, one of the constraints is that you want to fix the *current* car, not get a new one

The constraints could also include the

- overall cost of the repairs,
- how long the repair will take,
- that no new tools must be purchased

# Definition of problem solving

For this course, we will be operating under the definition of

> Writing an original program that performs a particular set of tasks and meets all stated constraints

# The fox, the goose, the corn

> A farmer with a fox, a goose, and a sack of corn needs to cross a river. The farmer has a rowboat, but there is room for only the farmer and one of his three items. Unfortunately, both the fox and the goose are hungry. The fox cannot be left alone with the goose, or the fox will eat the goose. Likewise, the goose cannot be left alone with the sack of corn, or the goose will eat the corn. How does the farmer get everything across the river?

# The key

You can take something back

# What if the possibility was made explicit

The farmer has a rowboat and it can be used to transfer items in either direction

> if you are unaware of all possible actions you could take, you may be unable to solve the problem

# Restating in more formal terms

First we'll list the constraints

1. The farmer can take only one item at a time in the boat
2. The fox and goose cannot be left alone on the same shore
3. The goose and corn cannot be left alone on the same shore

If we *remove* any of these constraints, the problem becomes easy

# Restating in more formal terms

Then we list possibilities

We could list the actions, that we think we can take

1. Operation: Carry the fox to the far side of the river
2. Operation: Carry the goose to the far side of the river
3. Operation: Carry the corn to the far side of the river

Remember that the **goal** of restating is to gain *insight* for a solution

One way of doing that is to make the operations more **generic**

1. Operation: Row the boat from one shore to the other.
2. Operation: If the boat is empty, load an item from the shore.
3. Operation: If the boat is not empty, unload the item to the shore.

Thinking *about the problem* may be as productive or **more productive** than thinking about the solution

# Sliding puzzle again

> A 3×3 grid is filled with eight tiles, numbered 1 through 8, and one empty space. Initially, the grid is in a jumbled configuration. A tile can be slid into an adjacent empty space, leaving the tile's previous location empty. The goal is to slide the tiles to place the grid in an ordered configuration, from tile 1 in the upper left.

# Sliding puzzle

The main difficulty of this puzzle is that it requires a *long chain of operations* in order to solve even partially

- A series of sliding operations may move some tiles to their correct final positions while moving other tiles out of position,
- or it may move some tiles closer to their correct positions while moving others farther away.

Because of this, it's difficult to tell whether any particular operation would *make progress* toward the ultimate goal.

Without being able to *measure progress*, it's difficult to formulate a strategy

# Reducing the size of the problem

Many who try a sliding tile puzzle simply move the tiles around randomly, hoping for luck

Let’s look at a smaller smaller grid to illustrate one of the possible strategies

> A 2×3 grid is filled with five tiles, numbered 4 through 8, and one empty space. Initially, the grid is in a jumbled configuration. A tile can be slid into an adjacent empty space, leaving the tile’s previous location empty. The goal is to slide the tiles to place the grid in an ordered configuration, from tile 4 in the upper left.

| | | |
|---|---|---|
| 6 | 8 | |
| 5 | 4 | 7 |

# The train method

A *train* is a sequence of tiles that are already in correct order, snaking through the grid

**Strategy:**

1. Pick the first unsolved tile
2. Move tiles to create a clear path
3. Slide the target tile into position
4. Repeat - each new tile "hitches" to the train
5. The train grows until all tiles are placed

The key insight: once a row or column is in position, you now have a smaller problem

# Inability to *plan a complete* solution does not prevent us from making strategies to systematically solve the puzzle

It’s better to develop a strategy than to use trial and error

# Problems are sometimes *divisible* in ways that aren't obvious

Even if you can't divide it, looking for that division might lead to insights about the problem

# Sudoku

A very old game, formally:

> A 9×9 grid is partially filled with single digits (from 1–9), and the player must fill in the empty squares while meeting certain constraints: In each row and column, each digit must appear exactly once, and further, in each marked 3×3 area, each digit must appear exactly once.

Sudoku puzzles vary in difficulty, primarily determined by the number of squares left and the types of techniques you're expected to know

And those constraints dictate where you should start

# Look for the *most constrained* part of the problem

These inherently simplify a problem since they limit the possible choices

# An example

> Suppose a group of coworkers wants to go to lunch together, and they've asked you to find a restaurant that everyone will like. The problem is, each of the coworkers imposes some kind of constraint on the group decision: Pam is a vegetarian, Todd doesn't like Chinese food, and so on

# Available restaurants

| Restaurant | Cuisine | Vegetarian | Price |
|---|---|---|---|
| The Green Plate | American | Yes | $$ |
| Dragon Palace | Chinese | No | $ |
| Pizza Roma | Italian | Yes | $$ |
| Sakura Sushi | Japanese | Yes | $$$ |
| El Rancho | Mexican | Yes | $ |
| Curry House | Indian | No | $$ |
| Burger Town | American | No | $ |
| Garden Cafe | American | Yes | $$ |

# Coworker constraints

- Pam – vegetarian options only
- Todd – no Chinese food
- Alice – budget under $$$
- Bob – no Indian food
- Carol – prefers American cuisine
- Dave – no sushi

> Which restaurant satisfies everyone?

# A test of skill (The Quarassi Loock)

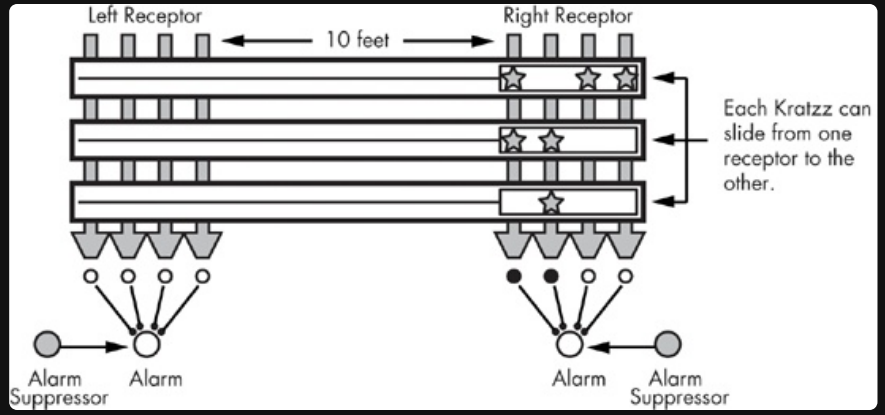
# The Quarassi Lock

A hostile alien race, the Quarrasi, has landed on Earth, and you've been captured. You've managed to overpower your guards, even though they are enormous and tentacled, but to escape the (still grounded) spaceship, you have to open the massive door. The instructions for opening the door are, oddly enough, printed in English, but it's still no piece of cake. To open the door, you have to slide the three bar-shaped Kratzz along tracks that lead from the right receptor to the left receptor, which lies at the end of the door, 10 feet away.

That's easy enough, but you have to avoid setting off the alarms, which work as follows. On each Kratzz are one or more star-shaped crystal power gems known as Quinicrys. Each receptor has four sensors that light up if the number of Quinicrys in the column above is even. An alarm goes off if the number of lit sensors is ever exactly one. Note that each receptor's alarm is separate: You can't ever have exactly one sensor lit for the left receptor or for the right receptor. The good news is that each alarm is equipped with a suppressor, which keeps the alarm from sounding as long as the button is pressed. If you could press both suppressors at once, the problem would be easy, but you can't since you have short human arms rather than long Quarassi tentacles.

Given all of this, how do you slide the Kratzz to open the door without activating either alarm?

# The Quarassi Lock



# Recognizing Analogies means avoiding most of the work

The more puzzles/programming problems/math problems/etc you solve, the more likely you'll find yourself in a position where you can build upon an existing solution

# General Problem Solving Techniques

# **Always** have a plan

You **must** always have a plan, even if you the plan gets abandoned, even if the plan leads to an incorrect solution

> I have always found that plans are useless, but planning is indispensable - General Dwight D. Eisenhower

Even if the problem does not survive first contact with the enemy, (discarded as soon as you type code), you **must** have a plan

*Do not hope for luck*

Planning also allows you to set *intermediate* goals and achieve them

Most programs don't do anything useful until they are close to completion, having intermediate goals helps your momentum

# **Restate** the problem

Restating the problem can produce *valuable results*. In some cases, it makes difficult problems seem easy

An analogy is circling the base of a hill that you must climb, before starting to climb

Even if a restatement **doesn't** lead to any immediate insight, it can help in other ways.

For example, if a problem has been assigned to you, you can take your restatement to the person who assigned the problem and *confirm your understanding*.

Also, restating the problem may be a *necessary prerequisite* step to using other common techniques, like reducing or dividing the problem.

# **Divide** the problem

Finding a way to divide a problem into steps or phases can make the problem much easier.

If you can divide a problem into two pieces, you might think that each piece would be *half as difficult* to solve as the original whole, but usually, it's *even easier* than that

> Suppose that you have 100 files you need to place in alphabetical order

Compare the difficulty of sorting 1 file between 99 other files

And sorting 1 file between 25 files, then arranging those files

Dividing a problem can often *lower the difficulty by an order of magnitude*

# Start with **what you know**

You should try to start with *what you already know* how to do and work outward from there.

Once you have divided the problem up into pieces, go ahead and complete any pieces you already know how to code.

Having a *working partial solution* may spark ideas about the rest of the problem

When we begin our investigation of a problem by applying the skills *we already have*, we may learn more about the problem and its ultimate solution.

# **Reduce** the Problem

When faced with a problem you are unable to solve, you reduce the scope of the problem, by either adding or removing constraints, to produce a problem that you do know how to solve

> Suppose you are given a series of coordinates in *three-dimensional* space, and you must *find the coordinates that are closest to each other*

You could reduce the problem to be *two-dimensional*, maybe even *one-dimensional*

Or

Only only have *three* three-dimensional values

*Reduction* allows us to work on a **simpler problem** even when we can't find a way to divide the problem into steps

# Sidestory

Beginning programmers often need to seek out experienced programmers or AI for assistance, but this can be a frustrating experience for everyone involved if the struggling programmer is unable to accurately describe the help that is needed.

> “Here’s my program, and it doesn’t work. Why not?”

Using the problem-reduction technique, one can **pinpoint** the help needed,

Saying something like,

> “Here’s some code I wrote.
> As you can see, I know how to find the distance between two three-dimensional coordinates, and I know how to check whether one distance is less than another.
> But I can’t seem to find a general solution for finding the pair of coordinates with the minimum distance.”

# Look for **Analogies**

An analogy, for our purposes, is a *similarity* between a current problem and a problem already solved that can be exploited to help solve the current problem.

Sometimes it means the two problems are *really the same problem*.

> This is the situation we had with the fox, goose, and corn problem and the Quarrasi lock problem.

Most analogies are *not that direct*.

Sometimes the similarity concerns only part of the problems.

For example, two number-processing problems might be *different in all aspects* except that both of them work with numbers *requiring more precision* than that given by built-in floating point data types

> You won't be able to use this analogy to solve the whole problem, but if you've already figured out a way to handle the extra precision issue, you can handle that same issue the same way again.

# Look for **Analogies**

Although recognizing analogies is the most important way you will improve your *speed and skill at problem solving*, it is also the **most difficult** skill to develop.

You can't look for analogies until you have a *storehouse* of previous solutions to reference.

This is where developing programmers often try to take a shortcut, stackoverflow, tutorials, and AI

For several reasons, though, this is a *mistake*.

1. If you don't complete a solution yourself, it's difficult to fully *understand* and *internalize* it.

You **don't need** to have written code to fully understand, but if you *could not have written the code*, your understanding will be necessarily limited.

2. Every successful program you write is *more than a solution* to a current problem

It's a potential source of analogies to solve future problems.

The more you rely on other programmers' code now, the more you will have to rely on it in the future

# Experiment

Sometimes the best way to make progress is to **try** things and observe the results.

Note that experimentation is **not** the same as guessing.

When you guess, you type some code and *hope* that it works, having no strong belief that it will.

An experiment is a *controlled* process.

You *hypothesize* what will happen when certain code is executed, try it out, and see whether your hypothesis is correct

Especially helpful when dealing with APIs or class libraries

## Relax

Don’t get frustrated. When you are frustrated, you won’t think as clearly, you won’t work as efficiently, and everything will take longer and seem harder